ConnectSphere Auth Service

Detailed explanation of how the `auth-service` works, how its classes connect to each other, how requests move through the system, and how security, tokens, persistence, validation, and error handling fit together in one complete flow.

MODULE	FRAMEWORK	RUNTIME GOAL
`com.connectsphere:auth-service`	Spring Boot 3.3.5, Spring Security, Spring Data JPA	User identity, login, JWT lifecycle, profile operations, search, and account state management

1. Purpose of the Service

The auth service is the identity and security foundation for the whole ConnectSphere platform. Its job is not only to register users and log them in, but also to make sure every protected request can be trusted. In practice, it handles user creation, password hashing, login verification, access-token generation, refresh-token rotation, token revocation during logout, profile updates, password changes, user search, and account deactivation.

Other microservices can rely on this service because it exposes a token validation endpoint and because the JWT itself carries the identity information needed by downstream consumers: user id, email, username, role, provider, token type, issuer, and expiry.

2. High-Level Architecture

```
Client
  |
  v
AuthResource (REST controller)
  |
  v
AuthServiceImpl (business rules)
  |
  +--> UserRepository ---------> users table
  |
```

```
  +--> PasswordEncoder --------> bcrypt hashes
  |
  +--> AuthenticationManager --> Spring Security credential check
  |
  +--> JwtTokenService --------> JWT creation / parsing / revocation
                                  |
                                  +--> RevokedTokenRepository --> revoked_tokens table

Incoming protected requests also pass through:

Client
  |
  v
SecurityFilterChain
  |
  v
JwtAuthenticationFilter
  |
  v
CustomUserDetailsService
  |
  v
UserRepository
```

3. Package Structure and Responsibilities

Package	Main Classes	Responsibility
`config`	`SecurityConfig`, `JwtProperties`	Defines security rules, authentication beans, password encoding, and JWT configuration values.
`controller`	`AuthResource`, `GlobalExceptionHandler`	Exposes REST endpoints and converts exceptions into clean HTTP responses.
`dto`	Request and response records	Defines the API contract between client and server.
`entity`	`User`, `RevokedToken`, `Role`, `AuthProvider`	Represents the domain model stored in the database.

Package	Main Classes	Responsibility
`repository`	`UserRepository`, `RevokedTokenRepository`	Handles database interaction using Spring Data JPA.
`security`	`JwtAuthenticationFilter`, `JwtTokenService`, `CustomUserDetailsService`, `AuthenticatedUser`	Handles authentication, JWT parsing, and the Spring Security user model.
`service`	`AuthService`, `AuthServiceImpl`	Contains the core business logic for all auth operations.

4. Domain Model

4.1 User Entity

The `User` entity is the central table for auth and profile data. It stores both security fields and user-facing profile information.

Field	Meaning	Why It Matters
`userId`	Primary key, UUID string	Used as stable identity across services and inside JWT subject claims.
`username`	Public handle	Must be unique so mentions, search, and profile addressing remain consistent.
`email`	Login identity	Used for authentication and lookup by Spring Security.
`passwordHash`	Stored bcrypt hash	The raw password is never saved; only the hash is persisted.
`fullName`, `bio`, `profilePicUrl`	Profile fields	Returned by profile endpoints and shown in the UI.
`role`	`USER` or `ADMIN`	Becomes a Spring Security authority and is embedded in JWT claims.
`provider`	`LOCAL`, `GOOGLE`, `GITHUB`	Supports future OAuth-based login flows.

Field	Meaning	Why It Matters
`active`	Account state flag	Inactive accounts cannot authenticate or refresh tokens.
`createdAt`, `updatedAt`	Audit timestamps	Auto-maintained via JPA lifecycle hooks.

The entity also contains `@PrePersist` and `@PreUpdate` methods. These methods automatically assign the UUID and timestamps, which keeps service logic simpler and makes persistence behavior consistent.

4.2 RevokedToken Entity

JWT is normally stateless, but logout requires a way to invalidate tokens before they naturally expire. That is why this service stores revoked token ids in the `revoked_tokens` table. When a token is logged out or rotated, its `jti` value is persisted with its expiry time. Later, any token check can reject that token.

5. Configuration Layer

5.1 Application Configuration

`application.yml` sets the default runtime behavior:

- Service port is `8081`.
- Default database is in-memory H2 for fast local development and testing.
- JPA `ddl-auto` is set to `update`, so the schema is created automatically.
- JWT issuer, secret, access-token TTL, and refresh-token TTL are centrally configured.
- Actuator health and info endpoints are exposed.

`application-mysql.yml` switches the service to MySQL, and `application-oauth.yml` enables Google and GitHub OAuth client registration only when the `oauth` profile is used.

5.2 SecurityConfig

`SecurityConfig` is where the service-wide security rules are assembled:

- CSRF is disabled because this is a stateless JSON API, not a session-based MVC form flow.
- Session creation is disabled using `SessionCreationPolicy.STATELESS`.

- Public endpoints are allowed for register, login, refresh, validate, search, health, and OAuth callback routes.
- All other routes require authentication.
- A `DaoAuthenticationProvider` connects Spring Security to the custom user loader and bcrypt encoder.
- `JwtAuthenticationFilter` is inserted before `UsernamePasswordAuthenticationFilter`.

> The service intentionally exposes both `/auth/**` and `/api/v1/auth/**`. This satisfies the original case-study endpoint naming and the versioned-API non-functional requirement at the same time.

6. Repository Layer

The repository layer is thin by design. It should only answer data-access questions and should not contain business rules. Those rules stay in the service layer.

6.1 UserRepository

- `findByEmail` is used by login, profile access, token validation, and Spring Security user loading.
- `findByUsername` helps enforce unique usernames.
- `existsByEmail` and `existsByUsername` support uniqueness checks.
- `findAllByRole` exists for admin-oriented extension points.
- `searchByUsername` uses JPQL to search by username or full name.
- `deleteByUserId` is present for future hard-delete or admin cleanup use cases.

6.2 RevokedTokenRepository

- Stores revoked token ids.
- `deleteByExpiresAtBefore` allows the service to purge revocation entries that are no longer needed.

7. Security Layer Internals

7.1 AuthenticatedUser

Spring Security works with `UserDetails`, not directly with the JPA entity. `AuthenticatedUser` is a lightweight adapter around `User`. It exposes:

- `getUsername()` as the email, because email is the login identifier.
- `getPassword()` as the stored bcrypt hash.
- `getAuthorities()` as a single role authority like `ROLE_USER`.
- `isEnabled()` and `isAccountNonLocked()` based on the `active` flag.

7.2 CustomUserDetailsService

When Spring Security wants to authenticate an email/password login, it asks the `UserDetailsService` to load a user by username. In this service, "username" means email. `CustomUserDetailsService` therefore looks up the user with `userRepository.findByEmail(...)` and wraps the result in `AuthenticatedUser`.

7.3 JwtTokenService

`JwtTokenService` is responsible for all token mechanics:

- Converts the configured secret into a signing key.
- Generates access and refresh tokens with different `tokenType` claims and different TTLs.
- Parses claims from existing tokens.
- Checks whether a token is expired, malformed, wrong type, or revoked.
- Revokes tokens by storing their token ids in the database.

The generated JWT contains:

- `sub` = user id
- `iss` = configured issuer
- `exp` = expiry time
- `iat` = issued time
- `jti` = unique token id
- custom claims = email, username, role, provider, token type

7.4 JwtAuthenticationFilter

This filter runs on every incoming request before the normal username/password filter. Its job is to check whether the request already carries a valid JWT. If it does, the filter builds the

authenticated security context so the controller can trust `Principal`.

```
1. Read Authorization header
2. Check that it starts with "Bearer "
3. Extract raw token
4. Ask JwtTokenService if it is usable as an ACCESS token
5. Extract the email claim
6. Load user details from the database
7. Reject if the account is inactive
8. Build UsernamePasswordAuthenticationToken
9. Store it in SecurityContextHolder
10. Continue the filter chain
```

8. Service Layer - Core Business Logic

`AuthServiceImpl` is the real center of the module. This is where validation, persistence, security, and token logic are combined into business operations.

Dependencies Used by AuthServiceImpl

- `UserRepository` for user reads and writes
- `PasswordEncoder` for hashing and password checks
- `AuthenticationManager` for Spring Security login validation
- `JwtTokenService` for token creation, validation, and revocation

Why This Layout Works

- Controller stays thin
- Security concerns stay reusable
- Repositories stay storage-focused
- Business rules live in one place

8.1 Register Flow

```
Client -> POST /auth/register
       -> AuthResource.register()
       -> AuthServiceImpl.register()
          -> ensureUniqueUser()
             -> UserRepository.findByEmail()
             -> UserRepository.findByUsername()
          -> verify role == USER
          -> verify provider == LOCAL
```

```
          -> PasswordEncoder.encode()
          -> UserRepository.save()
          -> buildAuthResponse()
             -> JwtTokenService.generateAccessToken()
             -> JwtTokenService.generateRefreshToken()
       -> return AuthResponse
```

Important rules inside registration:

- Only self-service `USER` registration is allowed.
- Registration is local only for now; OAuth users are expected from a future provider callback flow.
- Email is normalized to lowercase.
- Password is stored only as a bcrypt hash.

8.2 Login Flow

```
Client -> POST /auth/login
       -> AuthResource.login()
       -> AuthServiceImpl.login()
          -> AuthenticationManager.authenticate()
             -> DaoAuthenticationProvider
             -> CustomUserDetailsService
             -> PasswordEncoder.matches()
          -> getUserByEmail()
          -> reject if user is inactive
          -> buildAuthResponse()
       -> return new access + refresh tokens
```

The service intentionally uses Spring Security for credential verification instead of writing password-comparison logic by hand. That keeps authentication behavior aligned with the standard Spring pipeline.

8.3 Logout Flow

```
Client -> POST /auth/logout
       -> AuthServiceImpl.logout()
          -> JwtTokenService.revoke(accessToken)
          -> JwtTokenService.revoke(refreshToken) if present
          -> JwtTokenService.purgeExpiredRevocations()
```

Because JWT is stateless, logout would otherwise do nothing. The revocation table solves that problem by keeping a denylist of token ids until each token's expiry time passes.

8.4 Refresh Flow

```
Client -> POST /auth/refresh
       -> AuthServiceImpl.refreshToken()
          -> JwtTokenService.isRefreshTokenUsable()
          -> JwtTokenService.extractEmail()
          -> getUserByEmail()
          -> reject if inactive
          -> revoke old refresh token
          -> buildAuthResponse()
       -> return rotated tokens
```

This is a rotating refresh-token design. Every successful refresh revokes the old refresh token and returns a new token pair. That improves safety after token leakage and makes logout/deactivation stronger.

8.5 Validate Token Flow

The token validation endpoint is useful for downstream services or API gateways that need a trusted answer from the auth service.

```
Client/Service -> POST /auth/validate
               -> AuthServiceImpl.validateToken()
                  -> JwtTokenService.isAccessTokenUsable()
                  -> UserRepository.findByEmail()
                  -> reject if inactive
                  -> build TokenValidationResponse(valid=true, userId, email, role,
expiresAt)
```

8.6 Profile Read and Update

Protected profile routes rely on the authenticated `Principal`. Because the JWT filter already loaded the user into the security context, the controller can trust `principal.getName()` as the authenticated email.

- `GET /auth/profile` reads the current user and maps the entity to `UserProfileResponse`.
- `PUT /auth/profile` updates username, email, full name, bio, and profile picture after uniqueness checks.

8.7 Change Password

Password change has two validations:

- The current password must match the stored hash.
- The new password must be different from the old one.

If both checks pass, the new password is hashed and saved.

8.8 Search Users

Search is intentionally public in this version. It queries username and full name through the repository-level JPQL method, then filters the results so only active users are returned.

8.9 Deactivate Account

Deactivation is a soft state change. The user row remains in the database, but the `active` flag is set to false. That flag then affects:

- Login eligibility
- Refresh-token eligibility
- JWT filter authentication success
- Search results
- Token validation response

9. Controller Layer - API Surface

Endpoint	Method	Protected?	What It Does
`/auth/register`	POST	No	Creates a new local user and immediately returns tokens.
`/auth/login`	POST	No	Authenticates credentials and returns a fresh token pair.
`/auth/logout`	POST	Yes in practice	Revokes access token and optional refresh token.
`/auth/refresh`	POST	No	Accepts a refresh token and rotates the token pair.
`/auth/validate`	POST	No	Returns whether an access token is valid and who it belongs to.
`/auth/profile`	GET	Yes	Returns the authenticated user's profile.

Endpoint	Method	Protected?	What It Does
`/auth/profile`	PUT	Yes	Updates the authenticated user's profile data.
`/auth/password`	PATCH	Yes	Changes password after verifying the current password.
`/auth/search`	GET	No	Searches active users by username or full name.
`/auth/deactivate`	PATCH	Yes	Soft-deactivates the authenticated account.
`/auth/users/by-email/{email}`	GET	Yes	Returns a user profile by email for internal or cross-service use.

10. DTO Layer - Why It Exists

The DTO layer keeps the API contract stable and decoupled from database entities. This is important because:

- The entity contains internal fields like `passwordHash` that must never be returned.
- Request validation belongs on API input objects, not directly on the entity.
- Different operations need different shapes, for example login versus profile update versus token validation.

Main DTO groups:

- Input DTOs: `RegisterRequest`, `LoginRequest`, `RefreshTokenRequest`, `ChangePasswordRequest`, `UpdateProfileRequest`, `TokenValidationRequest`, `LogoutRequest`
- Output DTOs: `AuthResponse`, `UserProfileResponse`, `UserSummaryResponse`, `TokenValidationResponse`, `ApiMessageResponse`

11. Error Handling

`GlobalExceptionHandler` gives the service a consistent API error shape:

- `NotFoundException` becomes HTTP 404.

- `BadRequestException`, `BadCredentialsException`, and constraint violations become HTTP 400.
- `AccessDeniedException` becomes HTTP 403.
- `MethodArgumentNotValidException` becomes a field-by-field validation map.

This means controller methods stay clean. They can focus on delegation while the advice layer handles failure translation.

12. End-to-End Connection Map

```
Register
  AuthResource -> AuthServiceImpl -> UserRepository
                                  -> PasswordEncoder
                                  -> JwtTokenService

Login
  AuthResource -> AuthServiceImpl -> AuthenticationManager
                                  -> CustomUserDetailsService
                                  -> UserRepository
                                  -> PasswordEncoder
                                  -> JwtTokenService

Protected request (/profile, /password, /deactivate)
  SecurityFilterChain -> JwtAuthenticationFilter -> JwtTokenService
                                                 -> CustomUserDetailsService
                                                 -> UserRepository
                       -> Controller uses Principal
                       -> AuthServiceImpl executes business logic

Logout / Refresh
  AuthServiceImpl -> JwtTokenService -> RevokedTokenRepository

Search
  AuthResource -> AuthServiceImpl -> UserRepository.searchByUsername()
```

13. Data Flow by Concern

13.1 Identity

User id and email are the two most important identity anchors. The database stores them, the JWT carries them, and the service re-checks them on secure operations.

13.2 Authentication

Email and password enter through login, are verified by Spring Security, and are then converted into a stateless JWT-based session model.

13.3 Authorization

Authorization is role-based. The role is stored in the user entity, mapped to a Spring authority, and copied into the token claims so downstream systems can inspect it if needed.

13.4 Revocation

Logout and refresh rotation write token ids into the revocation repository. This gives the service a hybrid model: stateless token usage with stateful invalidation when needed.

14. Test Coverage

The integration test class verifies the most important service behavior through real HTTP calls with `MockMvc`:

- Registration returns an access token and stores a searchable user.
- Login succeeds with valid credentials.
- Token validation returns `valid=true` for a good token.
- Refresh generates a new access token from a valid refresh token.
- Password change works on an authenticated route.
- Deactivation succeeds on an authenticated route.

The service was validated with a passing `mvn clean test` run.

15. Current Design Decisions and Limits

- Public registration is intentionally limited to `USER` role for safety.
- OAuth dependencies and profile-specific configuration are present, but the provider callback flow is not implemented yet.
- Logout uses token revocation instead of deleting a session, because the service is stateless.
- H2 is used by default for local work, but MySQL configuration is ready through a profile.
- Search is public and returns only active users.

- Deactivation is soft, not hard deletion.

16. Summary

In simple terms, the auth service works because each layer has one clear job:

- The controller receives and returns HTTP.
- The service enforces business rules.
- The repositories store and fetch data.
- The security layer authenticates requests and creates trusted principals.
- The JWT service turns login state into signed tokens and handles revocation.
- The configuration layer wires the whole module together.

That separation is what makes the service understandable, testable, and ready to support the rest of the ConnectSphere microservices.